



第四章 C51程序设计

刘剑锋

ljf@csu.edu.cn



主要内容

- 4.1 C51程序设计基础
- 4.2 C51数据类型
- 4.3 C51运算量
- 4.4 C51运算符和表达式
- 4.5 C51语句
- 4.6 C51语言中数组、指针、结构和联合
- 4.7 函数、库函数和预处理命令
- 4.8 C51程序设计
- 4.9 C51语言和汇编语言混合编程

§ 4.1 C51程序设计基础

C51语言特点

- ◆C51语言兼具高级语言和汇编语言的特点
- ◆C51语言简洁、运算符丰富、数据结构众多
- ◆C51采用结构化设计程序，程序可移植性好
- ◆生成目标代码效率高

C51语言程序结构

文件名*.c

```
#include <reg51.h>          /*预处理命令（不加分号）*/
int func-1（形参）；        /*函数类型声明 */
char func-2（形参）；
.....

unsigned char count;        //定义全局变量
void main(void)
{
    说明；
    语句1；
    语句2；
    func-1（ ）；           //调用函数
    .....
}

int func-1（形参）          //函数定义
{
    说明；
    语句；
}
```


C51的字符集、标识符与关键字

- ◆ C51的字符集:数字、大小写英文字母, 空格, 下划线等
- ◆ C51标识符 :字母、数字和下划线组成, 由字母或者下划线开头 如init_time()
- ◆ 关键字: 有固定名称和特定含义的特殊标识符

§ 4.2 C51数据类型

字符型：unsigned char型和signed char，存放一个字节的数
据

整型：unsigned int型和signed int，存放两个字节的数
据

长整型：unsigned long型和signed long型 存放四个字节的
数据

浮点型：float型

字节地址	3	2	1	0
浮点数内 容	SEEEEEEE	EMMMMMMM	MMMMMMMM	MMMMMMMM

S位符号位，E为阶码位，M为尾数的小数部分



位型 : bit型和sbit型 , 占用一位二进制位

寄存器型 : sfr和sfr16两种类型

sfr为字节型特殊功能寄存器类型, 占用一个字节单元

sfr16为双字节型特殊功能寄存器类型, 占用两个字节单元

§ 4. 3 C51运算量

C51常量

整型常量：二进制 例如—— 11110000

十进制 例如—— 123、-25

十六进制 例如—— 0x123

浮点型常量：[±]整数部分.小数部分e[±]指数部分

例如：123.456e-3、-1.23e2

字符型常量：例如—— ‘0’、‘a’、‘1’、‘!’、‘\n’

字符串常量：例如：“12ab34”、“CHINESE”

C51变量

变量定义一般格式为:

[存储种类] 数据类型 [存储器类型] 变量名1[=初值],

存储种类 : 动态 (auto)、外部 (extern)、静态 (static)
寄存器 (register)

auto动态变量 ——在函数或者复合语句内部定义的变量，动态变量只在函数被调用时，系统才给动态变量分配存储单元，函数执行结束时释放存储空间。

extern外部变量——要使用一个其它文件中定义的变量，该变量在函数体中要用**extern**说明。

static静态变量——静态变量在程序运行时始终占用存储单元

register寄存器变量 ——变量存放在CPU内部RAM的寄存器中，处理速度快。

c编译器编译时能自动识别使用频率高的变量，并自动将其作为寄存器变量，用户可以不必专门声明。

数据类型:

char型、int型、bit型、long型、float型

例如:

```
unsigned char name; //定义变量name为无符号字符型变量
```

```
int x; //定义变量x为带符号整型变量
```

```
bit y; //定义变量y为位型变量
```

存储器类型 ——指定变量的存储区域

存储器类型	存储区域	说明
data	内部RAM的00H~7FH区域	直接寻址的内部RAM，存取速度最快
bdata	内部RAM的20H~2FH区域	位寻址区、允许位和字节的混合访问
idata	内部RAM的00H~FFH区域	间接寻址访问，用@R0、@R1间接访问
pdata	外部RAM某一页0~FFH区域	用MOVX @R0、@R1间接访问
xdata	整个外部RAM 0~FFFFH区域	用MOVX @DPTR间接访问
code	64KB的程序存储器区域	用MOVC指令访问



变量名：变量名可以由字母、数字和下画线组成，
但第一个字符必须是字母或者下画线。

例如： `unsigned char x=0x12, y=0x34, z=0x56`

C51扩展数据类型的变量定义：C51程序设计中支持4种新的数据类型的变量定义，这4种数据类型分别为bit型、sbit型、sfr型和sfr16型

bit普通位变量：普通位变量用来定义存放在内部RAM可以位寻址区域的变量

格式：bit [存储器类型] 变量名

例如：bit data key_in;
bit bdata key_a;
bit idata key_b;

sbit可位寻址的特殊位变量：可位寻址的特殊位变量可以位于内部RAM的20H~2FH区域和SFR中可以位寻址的位

sbit的格式：sbit 位变量名=位地址

sbit 3种定义方法:

(1) 用于指定已定义的可位寻址的SFR或者20H~2FH单元的某一位

例如: `sbit F0=PSW^5;` //定义F0为PSW.5

`sbit F1=PSW^1;` //定义F1为PSW.1

例如: `unsigned char bdata display;`

`//在bdata区域定义无符号字符变量`

`display`

`sbit dis_on=display^0;` //定义dis_on为

`display.0`

`sbit dis_off=display^1;` //定义dis_off为

(2) 指定可以位寻址的地址单元的某一位

例如: `sbit dis_on = 0x20^0;`

`sbit dis_off = 0x20^1;`

`sbit F0 = 0xd0^5;`

(3) 直接指定可寻址的位地址

例如: `sbit F0 = 0xd5;`

`sbit dis_on = 0x0;`

sfr特殊功能寄存器变量

功能:sfr直接指定8位的特殊功能寄存器的直接地址

定义格式为: sfr 特殊功能寄存器名=绝对地址

例如: sfr P0 = 0x80;

sfr SP = 0x81;

sfr TCON = 0x88;

sfr TH0 = 0x8c;

sfr TL0 = 0x8a;



sfr16 16位特殊功能寄存器变量

//用sfr16定义16位的特殊功能寄存器的低端地址

定义格式为:

sfr16 16位特殊功能寄存器名 = 绝对地址的低端地址

例如: sfr16 DPTR=0x82;

C51绝对地址访问

1. 使用宏定义访问绝对地址

定义格式如下：

```
#include <absacc.h>
```

```
#define 变量名 DBYTE [绝对地址]
```

```
//在内部RAM中定义绝对地址字节
```

变量

```
#define 变量名 DWORD [绝对地址]
```

```
//在内部RAM中定义绝对地址字变
```

量

```
#define 变量名 CBYTE [绝对地址]
```

```
//在ROM中定义绝对地址字节变量
```



```
#define 变量名 XBYTE [绝对地址]
```

```
//在外部RAM中定义绝对地址字节变量
```

```
#define 变量名 XWORD [绝对地址]
```

```
//在外部RAM中定义绝对地址字变量
```

```
#define 变量名 PBYTE [绝对地址]
```

```
//在外部RAM中某一页中定义绝对地址字节变量
```

```
#define 变量名 PWORD [绝对地址]
```

```
//在外部RAM中某一页中定义绝对地址字变量
```

例4-1 用define定义绝对地址变量

```
#include <absacc.h>
```

```
#define PA8255 XBYTE[0X0000]
```

```
#define PB8255 XBYTE[0X0001]
```

```
#define PC8255 XBYTE[0X0002]
```

```
#define COM8255 XBYTE [0x0003]
```

```
COM8255=0x83;           //把控制字写入8255控制寄存器
```

```
PA8255=0x0f;           //把数据写入8255的PA口
```

```
unsigned char a1,a2;
```

```
a1=PB8255;             //把8255PB口的数据写入变量a1
```

```
a2=XBYTE[0x0002]
```

```
           //把外部RAM地址为0002H单元的数据送给变量a2
```


2. 使用关键字_at_指定绝对地址

格式 : [存储器类型] 数据类型 变量名_at_ 地址常数;

例4-2 使用关键字_at_对绝对地址访问

```
#include <reg52.h>
```

```
data unsigned char x1_at_0x30;
```

```
    //指定变量x1在内部RAM的30H单元
```

```
xdata unsigned char x2_at_0x3000;
```

```
main ( )  
{  
    x1=0x0f;  
    x2=0xf0;  
}
```

注意：使用_at_定义的变量必须为全局变量！

储存模式

SMALL模式:在SMALL模式下，缺省存储器类型时，默认变量存放在idata区域，即片内数据存储器00H~0FFH单元。

COMPACT模式:在COMPACT模式下，缺省存储器类型时，默认变量存放在pdata区域，即存放在片外RAM的低256B空间。

LARGE模式:在LARGE模式下，缺省存储器类型时，默认变量存放在xdata区域，即存放于片外RAM的64KB空间。

在程序中变量的存储模式由`#pragma`预处理命令来实现，函数存储模式可通过在函数定义时后面带存储模式来说明。如果没有指定，则系统都默认为SMALL模式。

例4-3 变量和函数的存储模式

```
#pragma small    //变量存储模式为SMALL模式
unsigned char a;
#pragma compact  //变量存储模式为COMPACT模式
unsigned char b;
int func1(int x1, int x2) large //函数存储模式为LARGE
模式
{
return (x1*x2);
}
```

4.4 C51运算符和表达式

4.4.1 算术运算符与算术表达式

双目运算符 : + (加)、- (减)、* (乘)、/ (除)、% (取余)

单目算术运算符: + (取正)、- (取负)、++ (自增1)、
-- (自减1)

4.4.2 逻辑运算符与逻辑表达式

1. && 逻辑与

2. || 逻辑或

3. ! 逻辑非

4.4.3 关系运算符与关系表达式

<（小于）、<=（小于等于）、>（大于）、>=（大于等于）、
==（等于）、!=（不等于）

关系表达式的一般格式为： 表达式1 关系运算符 表达式2

关系表达式的运算结果为逻辑值，要么为真（值为1），要么为假（值为0）

4.4. 4位操作运算符与位表达式

6种位运算符：&（按位与）、|（按位或）、^（按位异或）、~（按位取反）、<<（左移）、>>（右移）

例如：P0=P0&0xfe; //该语句的功能为把P0.0清0

例如：P0=P0|0x01; //该语句的功能为把P0.0置1

例如：P0=P0^0x01; //该语句的功能为把P0.0取反

例如：unsigned char a=0x7f, b;

b= ~ a;

执行结果为b=0x80

左移运算

其功能是将一个操作数左移若干位，高位溢出舍去，低位补0。

例如： `unsigned char a=0x3f, b;`

`b=a<<2;`

执行结果为 `b=0xfc`，相当于 `b=a*22`

右移运算

其功能是将一个操作数右移若干位，对于无符号的数高端
端

移入0，低端移出舍掉。如果是带符号的数，高端移入操作数的符号位，右端移出位被舍掉。

例如： `signed char a=-5, b;`

`b=a>>1;`

则执行结果为 `b=0xfd`

4.4.5 赋值运算符与赋值表达式

赋值表达式的一般格式为： 变量 = 表达式

复合赋值运算符： +=、 -=、 *=、 /=、 %=、 <<=、 >>=、 &=、 ^=、 |=

4.4.6 逗号运算符与逗号表达式

逗号表达式的一般形式为：

表达式1， 表达式2， 表达式3， ……， 表达式n

例如： a=1, b=2;

4.5 C51语句

赋值语句、if语句、switch语句、while语句、for语句、goto语句、break语句、continue语句

4.5.1. if语句

if语句有3种形式:

- (1) if (表达式) 语句;
- (2) if (表达式) 语句1;
 else 语句2;
- (3) if (表达式1) 语句1;
 else if (表达式2) 语句2;
 else if (表达式3) 语句3;

 else if (表达式n) 语句n;
 else 语句n+1;

4.5.2. switch语句

switch语句是直接处理多分支的选择语句，一般格式为：

```
switch (表达式)
```

```
case 常量表达式1: 语句1;
```

```
case 常量表达式2: 语句2;
```

```
.....
```

```
case 常量表达式n: 语句n;
```

```
default: 语句n+1;
```



4. 5. 3. while语句

while语句的一般形式为： while（表达式） 语句；

4. 5. 4. do-while语句

do-while语句的一般形式为：

do

语句；

while（表达式）；

4.5.5. for语句

for语句的一般形式为：

for（表达式1；表达式2；表达式3）语句；

4.5.6. goto语句、break语句和continue语句

(1) goto语句为无条件转移语句，它的一般形式为：

goto 标号；

(2) break语句只能用在switch语句和循环语句中。

在switch语句中可以用break语句跳出switch结构，使程序继续执行switch结构后面的一个语句；

在循环语句中，使用break语句从循环体中跳出循环，提前结束循环而执行循环结构下面的语句。



(3) continue语句用在循环体结构中，用于结束本次循环，跳过循环体中continue下面尚未执行的语句，直接进行下一次是否执行循环的判断。

4.5.7. return语句

return语句一般放在函数的最后位置，用于终止函数的执行，并控制程序返回调用该函数时所处的位置。

4.6 C51语言中数组、指针、结构和联合

4.6.1 数组

C51数组中定义一维数组的一般格式如下：

数据类型 [存储器类型] 数组名 [常量表达式];

例4-13 在内部RAM中定义一个一维数组来存放10个学生的成绩：

```
unsigned char data score[10];
```

```
score[10]={0x8, 0x18, 0x28, 0x38, 0x48, 0x58, 0x68, 0x7d, 0x6f,  
0x45};
```

```
//给数组赋值，赋值后 score[0]=0x8, score[1]=0x18,  
score[2]=0x28, score[3]=0x38, ……， score[9]=0x45。
```

4.6.2 指针

1、 指针变量必须先定义后使用，指针变量定义的一般形式为：

数据类型 [存储器类型1] * [存储器类型2] 指针变量名；

- 数据类型指该指针变量指向对象的数据类型
- 指针中放的是该变量的地址
- 存储器类型1是指指针变量指向的变量的存储器类型
- 存储器类型2是指指针的存储器类型

例4-15

```
char *p1;
```

//定义p1为指针变量，指向一个字符变量

```
char data *p2;
```

//定义p2为指针变量，指向一个内部RAM中字符变量

```
int data *p3;
```

//定义p3为指针变量，指向一个内部RAM中整型变量

```
int xdata * data p4;
```

//定义p4为指针变量，指向一个外部RAM中整型变量，

➤ 指针中存放指针变量地址，在内部RAM中占用2个字节

2、指针变量的引用

C51中的单目运算符&（取地址运算符），功能为取变量的地址

例4-16 应用例4-15中定义的变量，再定义了一些新的变量，将地址赋给指针。

```
char data x;           //定义x为data区字符型变量
int data y;            //定义y为data区整型变量
int xdata z;           //定义z为xdata区整形变量
char data a[5];        //定义a[5]为data区字符型变量
p1=&x;                 //取变量x的地址赋给指针p1，指针变量*p1指向变量x
p2=&a[0];              //取a[0]的地址赋给指针p2，指针变量*p2指向变量a[0]
p3=&y;                 //取变量y的地址赋给指针p3，指针变量*p3指向变量y
p4=&z;                 //取变量z的地址赋给指针p4，指针变量*p4指向变量z
p1++;                  //字符型变量占用一个字节，指针实际加1
p3++;                  //整型变量占用两个字节，指针实际加2
```

4.6.3 结构

结构的定义

结构类型的一般形式为：

```
struct 结构类型名
```

```
{
```

结构成员表

```
};
```

4.6.4 联合

1) 联合类型和变量一起定义的一般格式为:

```
union 联合类型名
```

```
{
```

```
    成员表
```

```
} 变量名表;
```

2) 若先定义联合类型，再定义联合类型变量，则一般格式为:

```
union 联合类型名
```

```
{
```

```
    成员表
```

```
};
```

```
union 联合类型名 变量名表;
```

4.6.5 枚举

枚举类型变量定义的一般格式如下：

```
enum 枚举名  
{  
    标识符1 [=整型常数1],  
    标识符2 [=整型常数2],  
    .....  
    标识符N [=整型常数N]  
} 枚举变量;
```

4.7 函数、库函数和预处理命令

4.7.1 函数的定义

定义一个函数的一般形式如下：

[函数数据类型] 函数名 (形参列表)

形参说明

{

 局部变量定义；

 语句；

}

有返回值的函数必须有一个或几个return语句

4.7.2 函数的调用和声明

1. 函数的调用

函数调用的一般形式为：函数名 （实参列表）

例如： `int a, b, c;`

`c=min(a, b);` //调用上节中的min函数，求a，b中的最小值

2. 函数的声明

函数声明的一般格式为：

[函数数据类型] 函数名 （形参列表）

若调用外部函数，则函数声明的一般格式为：

`extern` [函数数据类型] 函数名 （形参列表）

4.7.3 中断函数

1. 中断函数的定义

C51中用关键字interrupt和中断号定义中断函数，一般格式如下：

```
[void] 中断函数名( ) interrupt 中断号m [using n]
{
    函数体
}
```

中断号m的取值为0~5，分别对应51系列单片机中的6个中断源。外中断0（INT0）的中断号为0，定时计数器T0的中断号为1，外中断1（INT1）的中断号为2，定时计数器T1的中断号为3，串行口中断的中断号为4，定时计数器T2的中断号为5。



选项[using n]，指定中断函数中使用的工作寄存器组号，n的取值为0~3，表示可以选择中断函数中使用工作寄存器为0、1、2或者3区的工作寄存器。

4.7.4库函数

C51软件包的库包含标准的应用程序，也称为库函数，包括本征函数和非本征函数，一些特定功能的库函数包含在一个头文件中，如果需要使用某一库函数，必须在源程序中用预处理命令定义与该函数相关的头文件。

例如：

```
#include <intrins.h> //包含文件名为intrins.h的头文件  
# include <reg52.h> //包含文件名为reg52.h的头文件  
#include <ctype.h> //包含文件名为ctype.h的头文件
```

4.7.4库函数

1. 本征函数文件

本征函数文件名为intrins.h，文件中包含了常用的本征函数，本征函数也称为内部函数，一共有9种函数，这种函数不采用调用形式，编译时直接将代码插入当前行。

(1) 左环移本征函数

1) 函数名: `_crol_`

函数原型: `unsigned char _crol_ (unsigned char a, unsigned char n) ;`

函数功能: 将无符号字符型变量a，循环左移n位

2) 函数名: `_irol_`

函数原型: `unsigned int _irol_ (unsigned int a, unsigned char n) ;`

函数功能: 将无符号整型变量a，循环左移n位

3) 函数名: `_lrol_`

函数原型: `unsigned long _lrol_ (unsigned long a, unsigned char n) ;`

函数功能: 将无符号长整型变量a，循环左移n位

(2) 右环移本征函数

1) 函数名: `_cror_`

函数原型: `unsigned char _cror_ (unsigned char a, unsigned char n) ;`

函数功能: 将无符号字符型变量a, 循环右移n位

2) 函数名: `_iror_`

函数原型: `unsigned int _iror_ (unsigned int a, unsigned char n) ;`

函数功能: 将无符号整型变量a, 循环右移n位

3) 函数名: `_lror_`

函数原型: `unsigned long _lror_ (unsigned long a, unsigned char n) ;`

函数功能: 将无符号长整型变量a, 循环右移n位

(3) 其它本征函数

1) 函数名: `_nop_`

函数原型: `void _nop_(void)`

函数功能: 产生一条NOP空指令, 执行一次空操作

2) 函数名: `_testbit_`

函数原型: `bit _testbit_(bit x);`

函数功能: 产生一条JBC指令, 该函数测试一个位, 如果该位为1, 则将该位清0, 并且返回值为1, 否则返回值为0

例如: `if _testbit_(RI) a=SBUF; //若RI=1则清0, 并读`

`SBUF`

3) 函数名: `_chkfloat_`

函数原型: `unsigned char _chkfloat_(float x);`

函数功能: 检查浮点型变量x的状态, 返回值为无符号字符型数据,

其值可以为0、1、2、3、4, 返回值意义为:

0 标志浮点数

1 浮点数0

2 +INF正溢出

3 -INF负溢出

4 NaN 不是一个数的错误状态

2. 非本征函数

库函数中非本征函数在调用时由ACALL或者LCALL指令调用，常用的包含非本征函数的头文件有：

ctype.h	字符函数
stdio.h	一般I/O函数
string.h	字符串函数
stdlib.h	标准函数
math.h	数学函数
absacc.h	绝对地址访问宏定义
stdarg.h	变量参数表
setjmp.h	全程跳转
reg*.h	SFR定义文件

4.7.5 预处理命令

1. 宏定义命令#define

宏定义命令用来指定标识符的值，一般定义格式为：

`#define 标识符 数字或字符序列`

`#define 标识符 (形参) 数值或字符序列`

例如：

`#define PI 3.1415926`

宏定义后PI作为一个常量使用，预处理时将程序中的PI换成数值3.1415926。

`#define s(a,b) a*b`

`area=s(2,3);`

预处理时将area换成2*3。

2. 文件包含命令#include

包含命令的一般格式为：

`#include` <头文件名> 或者 `#include` “文件名”

例如：`#include math.h` //包含数学计算函数库的头文件

4.8 C51程序设计

4.8.1数值运算程序设计

1. 数制转换程序

例4-23十六进制到十进制转换

待转换的整型十六进制数存放于变量x中，编写C51程序，将变量x中的十六进制数转换为相应的十进制数，存放于数组a[]中，其中转换结果的个位、十位、百位、千位、万位分别存放于a[0]、a[1]、a[2]、a[3]、a[4] 中 。



```
void main ( )
```

```
{
```

```
    unsigned int x;
```

```
    unsigned char a[5]={0, 0, 0, 0, 0};
```

```
    unsigned char j, i=0;
```

```
    for (j=1; j<=5; j++)
```

```
    {
```

```
        a[i]=x%10;           //x除以10的余数存入a[i]
```

```
        x=x/10;             //x除以10的商（整数）
```

```
        i++;
```

```
    }
```

```
}
```

例4-24 十进制到十六进制的转换

十进制数以BCD码形式存放数组a[]中，a[0]、a[1]、a[2]、a[3]、a[4] 中分别存放个位、十位、百位、千位、万位，把此数转换为十六进制数存放于变量x中。

x初始化为0

$$\begin{aligned}x &= a_4 \times 10^4 + a_3 \times 10^3 + a_2 \times 10^2 + a_1 \times 10 + a_0 \\ &= (((((x \times 10 + a_4) \times 10 + a_3) \times 10 + a_2) \times 10 + a_1) \times 10 + a_0\end{aligned}$$



```
void main()
{
    unsigned int x=0;
    unsigned char a[5]={1, 2, 3, 4, 5};
    unsigned char j, i=4;
    for (j=1; j<=5; j++)
    {
        x=x*10+a[i];
        i--;
    }
}
```

2. 求最大值

例4-25 有八个无符号字符型数据放在数组a[]中，求其中最大值。

数组中最大值存放于变量max中，设置max中初值为0，把数组中的数逐一与max比较，若比max大则取代max中的值，若比max小则保持max中值不变。

```
void main()
{
    unsigned char i=0,max=0;
    unsigned char a[]={1, 2, 3, 4, 5, 100, 45, 56};
    while (i<8)
    {
        if (a[i]>max) max=a[i];
        i++;
    }
}
```

3. 查表程序

例4-26应用查表程序求0~5这几个数的平方值。

```
void main()
{
    unsigned char square, i;
    unsigned char a[]={0, 1, 4, 9, 16, 25};
        //几个数的平方值放在数组a[]中
    square=a[i];
        //i为数值，square为查表所得平方值
}
```

程序设计思路：把几个数的平方值预先放在数组中，根据序号查找相应数组中的数据，即为该序号的平方值。

4.8.2 硬件接口程序设计

例4-27 通过单片机的并行口P1口，控制8路LED灯轮流点亮。
引脚为0时灯亮，为1时灯熄灭。

```
#include <reg51.h>
#include <intrins.h>
void delay( );           //延时函数声明
void main()
{
    P1=0xfe;
    while (1)
    { delay ( );          //调用延时程序
      P1=_crol_(P1, 1);    //P1数据左移1位
    }
}
void delay ( )           //延时函数
{
    unsigned char i, j;
    for(i=0; i<0xff; i++)
        for(j=0; j<0xff; j++);
}
```

例4-28 单片机P1口控制8路LED灯，由两端往中间亮，再由中间往两端亮，如此反复循环。为0时灯亮，为1时灯熄灭。

```
#include <reg51.h>
#include <intrins.h>
void delay( );
unsigned char a[]={0x7e,0xbd,0xdb,0xe7,0xdb,0xbd};
unsigned char i=0;
void main()
{
    while (1)
    { if (i==6) i=0;
      P1=a[i];
      delay ( );
      i++;
    }
}
void delay ( )
{
    unsigned char i, j;
    for(i=0;i<0xff;i++)
        for(j=0;j<0xff;j++);
}
```

4.9 C51语言和汇编语言混合编程

4.9.1. 在C51语言中嵌入汇编语言

在C51语言中嵌入汇编语言的一般格式如下：

```
#pragma asm
```

```
    Assembler Code Here
```

```
    //汇编程序代码
```

```
#pragma endasm
```


例4-29在C51语言中插入一段软件延时程序，C51语言文件名为main.c。

```
#include <reg51.h>
void main(void)
{
    P2=1;
    #pragma asm                //嵌入汇编语言程序段，实现精确延时
    MOV R7, #10
    DEL:MOV R6, #20
    DJNZ R6, $
    DJNZ R7, DEL
    #pragma endasm
    P2=0;
}
```

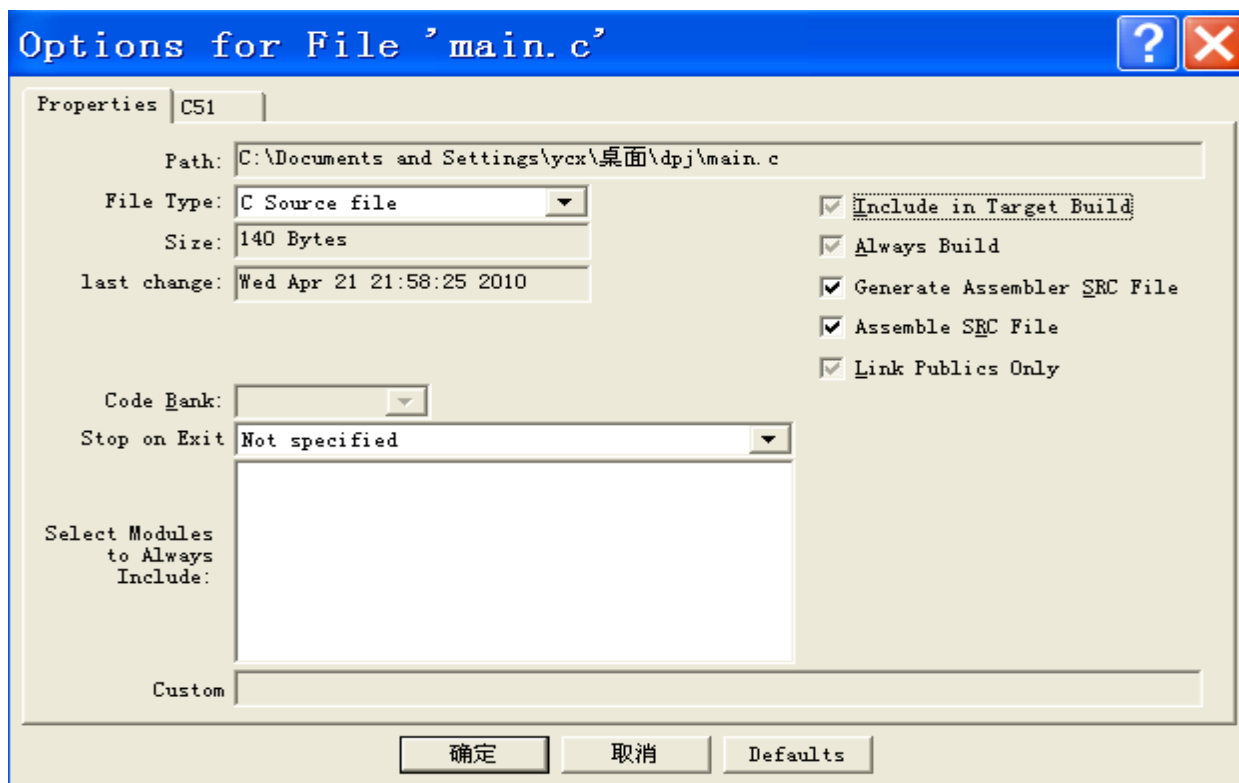
上述程序中，也可以采用函数调用方式，在C51语言中调用汇编函数，程序如下：

```
#include <reg51.h>
void delay(void);           //delay函数声明
void main(void)
{
    P2=1;
    delay();                //调用延时函数
    P2=0;
}

void delay()
{
    #pragma asm
    MOV R7, #10
    DEL: MOV R6, #20
    DJNZ R6, $
    DJNZ R7, DEL
    #pragma endasm
}
```

用keil软件编译程序时，还需要注意以下几点：

- (1) 新建一个项目，并在项目中添加文件main.c。
- (2) 在keil软件的左边工程窗口，右键main.c单击“Options for File ‘main.c’”选项，在该对话框中，选中Generate Assembler SRC File和Assemble SRC File两个选项，默认情况下，这两个选项前面的选项是灰色的钩，单击两下就变成黑色的钩，即选中状态，如图下所示。





(3) 在项目中添加最后一个文件，如为 Small模式，
将 Keil\C51\Lib\C51S.Lib加入项目中，如为Compact模式，
将C51C.Lib加入项目中，如为Large模式，将C51L.Lib加入项目中，并将此文件作为项目中的最后一个文件。

(4) 编译项目。

4.9.2. C51语言和汇编语言程序参数的传递

keil C51编译器使用寄存器传递参数非常方便，但是使用寄存器传递参数最多只能传递3个参数，并且入口参数选择固定的寄存器。入口参数中第一个参数、第二个参数和第三个参数占用寄存器如下表所示。

参数传递的寄存器选择

参数数据类型	char	int	long、 float	一般指针
第1个参数	R7	R6、R7	R4~R7	R1、R2、R3
第2个参数	R5	R4、R5	R4~R7	R1、R2、R3
第2个参数	R3	R2、R3	无	R1、R2、R3

例如：在C51语言中调用汇编函数func1(char x, int y)，汇编函数名为func1，其中入口参数有两个，第1个参数为 char型参数，此参数存放与寄存器R7中，第2个参数为int型参数，存放与寄存器R4、R5中，其中高位字节在R4中，低位字节在R5中。

汇编函数的返回值，通过寄存器或存储器传递出口参数给C51程序，具体见表4-6。

汇编函数返回值寄存器

返回值	寄存器	说明
bit	c	进位标志
char	R7	
int	R6、R7	高位在R6，低位在R7
long	R4~R7	高位在R4，低位在R7

4.9.3. 带参数传递的汇编语言调用程序实例

例4-30 编程使得单片机P1.0引脚输出周期为4ms的方波，同时P1.1引脚输出周期为8ms的方波。

编程思路：

- 用C51语言编写主程序a1.c，调用延时4ms的延时程序，使P1.1引脚输出周期为8ms的方波；
- 用C51语言编写a2.c的文件，通过调用延时程序使P1.0引脚输出周期为4ms的方波，即高电平和低电平时间持续2ms，延时2ms由调用汇编函数delay1ms（）实现；
- 文件a3.a51中即是用汇编语言编写的延时程序。

具体程序如下：

C51语言文件a1.c

```
#include <reg51.h>
```

```
#define uchar unsigned char
```

```
sbit P1_1=P1^1;
```

```
extern void delay4ms(void);
```

//调用外部函数声明

```
main ( )
```

```
{
```

```
    for ( ; ; )
```

```
{
```

```
        P1_1=0;
```

```
        delay4ms( );
```

//调用延时4ms函数

```
        P1_1=1;
```

```
        delay4ms( );
```

```
    }
```

```
}
```


C51语言文件 a2.c

```
#include <reg51.h>
```

```
#define uchar unsigned char
```

```
sbit P1_0=P1^0;
```

```
extern delay1ms (uchar x);
```

```
    //外部函数delay1ms ( ) 声明，入口参数为x
```

```
void delay4ms(void)
```

```
{
```

```
    P1_0=0;
```

```
    delay1ms(2);
```

```
    //调用外部函数delay1ms (2) ， 参数2送入寄存器R7
```

```
    P1_0=1;
```

```
    delay1ms(2);
```

```
}
```

汇编程序文件 a3. a51

```
public _delay1ms
    //用public 声明_delay1ms为其它函数调用，以 “_”
    开头后跟函数名
    de segment code        //定义de段位再定位程序段
    rseg de                //选择de为当前段
    _delay1ms:nop
        dela: mov r1, #0f8h
        lop1:nop
            nop
                djnz r1, lop1
                djnz r7, dela
                ret
    end
```



用keil软件对程序进行编译、调试时，先新建一个新项目，加入上述三个文件，凡是c文件编译前选中Generate Assembler SRC File和Assemble SRC File两个选项，然后对项目进行编译和调试。



本章小结